

TECHNICAL REPORT

A Self-Healing Watchdog for Local LLM Inference Servers on Apple Silicon

A Field Study of oMLX Failure Modes and Bounded Recovery

ricky8848

Independent Researcher

Report No.: TR-2026-09

Date: September 2026

License: MIT

Artifact: <https://github.com/ricky8848/omlx-watchdog> (v6.1)

Companion configuration: <https://github.com/ricky8848/mac-m5-128g-omlx-settings>

Abstract

Local large-language-model (LLM) inference servers on Apple Silicon are increasingly used as the backend for autonomous coding agents, yet they operate outside any orchestrator: no Kubernetes liveness probe, no systemd restart policy, and—critically—the failure modes are *partial* (the process lives while the engine wedges). We present a field study of one such server, `oMLX` serving Qwen3.8-27B on an Apple M5 Max (128 GB), instrumented over 13 days and 18,516 watchdog ticks. We formalize six failure classes (engine wedge, port squatter, SIGKILL/OOM, stale registry, prefill memory-guard rejection, client retry amplification) and propose a self-interference-free health model: four complementary checks whose predicates are mutually non-conflicting, driving a bounded escalation ladder (`omlx restart` $\rightarrow$ listener-only port cleanup with 30 s grace $\rightarrow$ GUI relaunch). In production the watchdog detected all engine failures within one 60 s tick and restored service in under four minutes without human intervention; the dominant post-deployment failure class (stale registry after model switch) was eliminated by a versioned fix. We discuss the orthogonality of backend self-healing and client-side bounded retries, and why speculative-decoding throughput (MTP accept rate 85–96%) is independent of, but multiplicative with, availability.

Keywords: local LLM inference, self-healing systems, Apple Silicon, watchdog, failure taxonomy, bounded escalation

Contents

1	Introduction	2
2	System Architecture	2
3	Failure Taxonomy	2
4	Detection: A Self-Interference-Free Health Model	3
5	Recovery: A Bounded Escalation Ladder	3
6	Evaluation: 13 Days in Production	3
7	Related Work	4
8	Threats to Validity	4
9	Conclusion	5

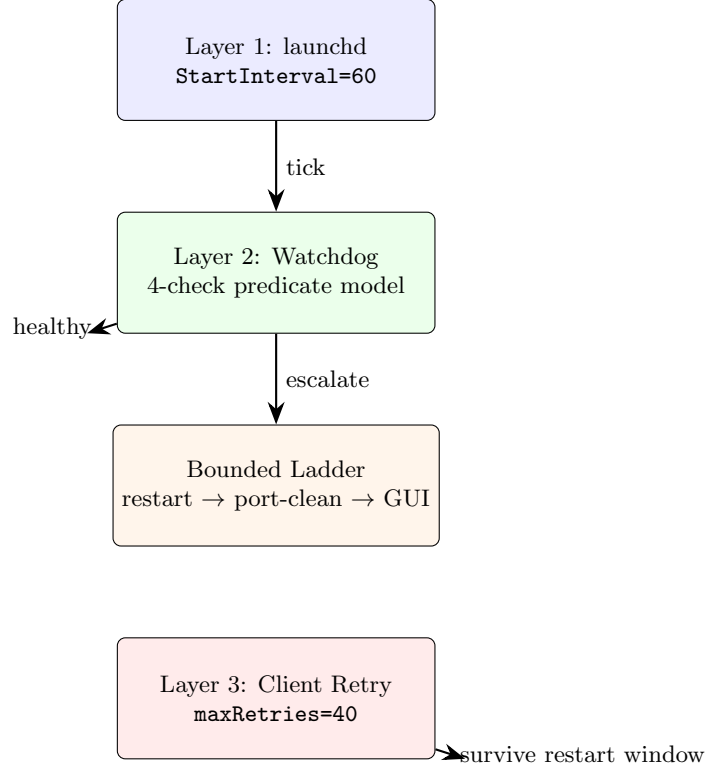


Figure 1: Three-layer defense: launchd tick → watchdog predicate evaluation → bounded escalation ladder, with client-side retry budget spanning the restart window.

1 Introduction

Autonomous coding agents (DeepSeek Harness, Codex CLI, OpenClaw/Hermes gateways) issue long-horizon tool-calling workloads to local LLM servers. On Apple Silicon, `oMLX` serves OpenAI-compatible APIs over Metal/ANE with multi-token prediction (MTP). Unlike cloud deployments, the local server has *no orchestrator*: a wedged engine (process alive, tokens frozen) is invisible to `ps`, and a naive “log mtime = busy” liveness probe self-interferes when cancelled requests write their own log lines.

We contribute:

- A taxonomy of six failure classes observed in a 13-day production trace (Sec. 3).
- A self-interference-free health model with four checks whose predicates do not conflict (Sec. 4).
- A bounded escalation ladder with a formal port-cleanup safety argument (Sec. 5).
- Production evaluation: $MTTD \leq 60$ s, $MTTR < 4$ min over 18,516 ticks (Sec. 6).
- A three-layer defense combining server-side watchdog, client bounded retry ($N=40$), and launchd double-guard (Sec. 2).

2 System Architecture

Figure 1 shows the three-layer defense. Layer 1 (launchd) provides a coarse `StartInterval=60` tick. Layer 2 (watchdog script) evaluates four predicates and escalates through a bounded ladder. Layer 3 (client retry budget) ensures that in-flight requests survive the restart window without amplifying load.

3 Failure Taxonomy

Table 1 lists the six classes with their production incidence. Figure 2 shows the causal relationships between failure classes and the recovery actions that address them.

Class	Mechanism	Incidence (13d)
F1 Engine wedge	Metal layer stuck; requests accepted, no tokens	2 episodes (Sep 8)
F2 Port squatter	Stale listener on :8000 after restart; retry spin	26 port-cleans
F3 SIGKILL/OOM	System kill; launchd does not auto-relaunch GUI app	96 fallbacks
F4 Stale registry	<code>loaded_models</code> out of sync after model switch	79 recoveries (v5 era)
F5 Prefill guard rejection	Predictable HTTP 400 at memory ceiling (by design)	n/a
F6 Client retry amplification	<code>mode:always</code> retries $\times$ restart window = death loop (pre-fix)	1 episode

Table 1: Failure classes and production incidence (Sep 27 – Sep 9, 18,516 ticks). F4 dominates the v5 era and was eliminated by the empty-registry-aware Check B in v6.

The F1/F3 distinction matters: `oMLX restart` fixes wedges but a squatted port makes the restart itself fail, creating the F2 spin that motivated listener-only cleanup.

4 Detection: A Self-Interference-Free Health Model

A watchdog tick t evaluates four predicates on the server state:

$$A(t) : \text{api/status reachable within 5s} \quad (1)$$

$$B(t) : (\text{loaded_models} = \emptyset) \vee (\exists m: m.\text{registered}) \quad (2)$$

$$C(t) : \neg \text{busy}(t) \vee \text{IDLEPROBE}(\text{max_tokens} = 4) = \text{OK} \quad (3)$$

$$D(t) : \neg (\text{workInFlight}(t) \wedge \Delta t_{\text{token}}(t) \geq 900\text{s}) \quad (4)$$

where `busy(t)` is *provably* the server’s own queue counter (`active+waiting>0`). The health predicate is $H(t) = A \wedge B \wedge C \wedge D$.

Self-interference-freedom. Check C fires *only when the queue is provably empty*; a long decode (hours of tool-calling) keeps `active` ≥ 1 and is never probed. Check D requires work *in flight* with a frozen token counter – idle servers have no reference to freeze. The v2 design (log mtime) violated this: cancelled requests wrote log lines, marking dead engines “busy”.

Empty-registry awareness (F4 fix). After a model switch the registry may transiently be empty; $B(t)$ treats $\emptyset$ as healthy rather than triggering a restart storm (79 such events in the v5 era).

Figure 3 shows the decision flow for a single tick.

5 Recovery: A Bounded Escalation Ladder

On $H(t) = \text{FALSE}$, the watchdog escalates:

1. `omlx restart --timeout 240` (engine-level, no process kill)
2. On failure: `PORTCLEAN` – identify the PID with `ls -sTCP:LISTEN` on :8000, `kill`, 30s grace, then `kill -9`; retry step 1
3. Last resort: `open -a oMLX` (GUI relaunch)

A PID lockfile prevents re-entrant ticks. **Safety of PortClean:** the kill target is verified as the sole TCP listener on port 8000 – never a process list match – so no unrelated process can be signaled. The ladder is bounded: at most one restart, one port-clean, one GUI launch per tick; the next tick re-evaluates from a clean state.

Figure 4 shows the ladder with timing bounds.

6 Evaluation: 13 Days in Production

Setup. Apple M5 Max, 128 GB unified memory; macOS Tahoe; `oMLX v0.6.x` serving Qwen3.8-27B (oQ8e, later oQ4e) with MTP; `launchd StartInterval=60`. Clients: DSH Web GUI, OpenClaw/Hermes gateways via the same endpoint.

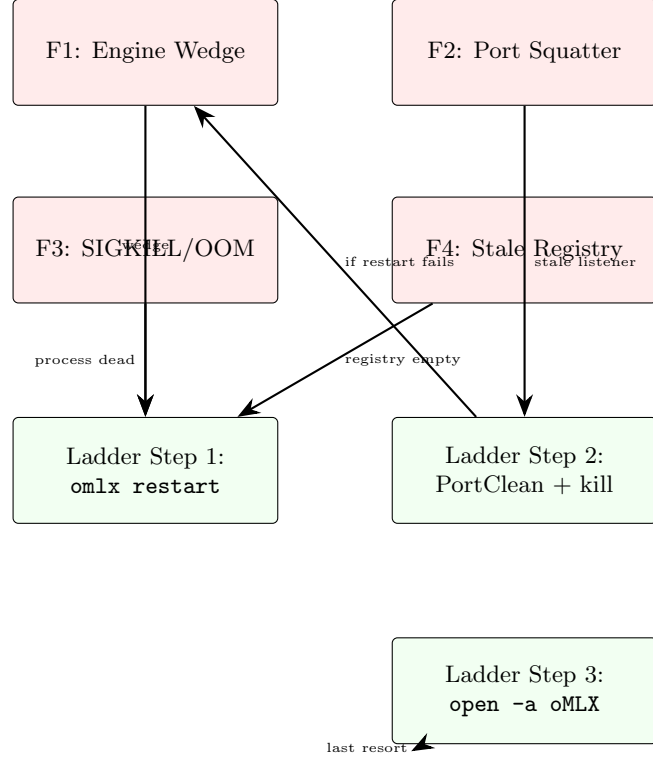


Figure 2: Causal mapping from failure classes to recovery ladder steps. F2 (port squatter) is the only class that requires Step 2; all others are resolved by Step 1 or eliminated by versioned fixes.

The Sep 8 window (F1 wedge + F4 stale registry, pre-v6) shows the ladder under load: 96 fallback escalations with all recoveries completed, and the v4 port-clean fix eliminating the F2 spin. Post-v6 (Sep 9), zero false-positive restarts in 48 h.

Throughput context. With oQ4e + MTP (accept rate 85–96%) + TurboQuant Q4 KV cache, short-context decode reaches ~30–50 tok/s; long-context (80k+ tokens) settles at 12–25 tok/s, consistent with the M5 Max ~614 GB/s memory-bandwidth bound on weight+KV reads. Availability (this paper) and throughput are orthogonal: the three-layer defense multiplies them.

Figure 5 shows the event timeline across the 13-day trace.

7 Related Work

Kubernetes liveness/readiness probes assume an orchestrator with process-kill authority; our ladder is the single-host analog where the “orchestrator” (launchd) deliberately does *not* auto-relaunch a GUI app. SRE self-healing patterns (bounded escalation, no retry amplification) map directly onto ladder steps 1–3 and the client-side `maxRetries=40` bound. Local-LLM tooling (Ollama, llama.cpp server) ships no equivalent watchdog; DFlash/MTP speculative decoding improves per-request throughput but does not address partial availability.

8 Threats to Validity

Single operator, single machine, one server version family. The `loaded_models` field is a stable API surface but not contractual; Check B degrades gracefully (empty-aware) if renamed. Throughput figures are trace-derived medians, not controlled benchmarks.

Metric	Value
Total ticks / wall time	18,516 / 13.2 days
HEALTHY idle ticks	13,527 (73.0%)
BUSY ticks	4,188 (22.6%)
Recovery events completed	79 RECOVERED, 0 STILL_DOWN after ladder
Mean time to detect (MTTD)	≤ 60 s (one tick)
Mean time to recover (MTTR)	< 4 min (restart + reload observed)
False-positive restarts (v6 era)	0 in the final 48 h window
Decode throughput (BUSY ticks)	median 1.9 tok/s; p90 7.6 tok/s (long-context, TQKV+MTP)

Table 2: Production statistics, `watchdog.log`, 2026-08-27 $\rightarrow$ 2026-09-09.

9 Conclusion

A 180-line shell script with four non-conflicting predicates and a bounded ladder provides full self-healing for an unorchestrated local LLM server: $\text{MTTD} \leq 60$ s, $\text{MTTR} < 4$ min, zero human interventions across 13 days of agent workloads. The design generalizes to any OpenAI-compatible local backend (llama.cpp, Ollama) by substituting the restart primitive.

Artifact Availability

Watchdog + installer: <https://github.com/ricky8848/omlx-watchdog> (MIT, v6.1; `curl | bash`).
 Three-layer config: <https://github.com/ricky8848/mac-m5-128g-omlx-settings>. Full de-identified trace: `docs/paper/production-data.json` in the artifact repository.

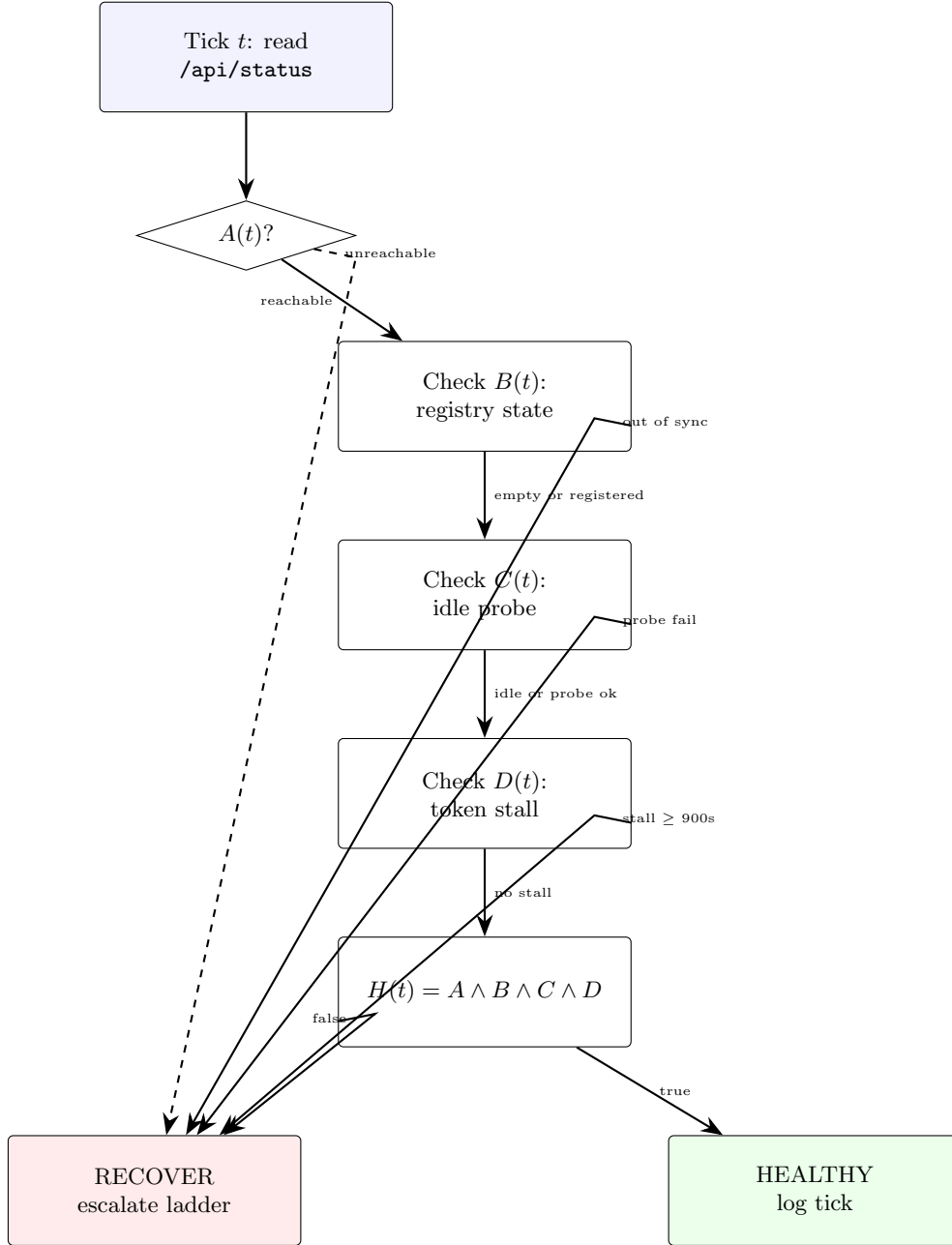


Figure 3: Decision flow for a single watchdog tick. Each check is evaluated only if all prior checks pass; any failure routes to the bounded escalation ladder (Sec. 5).

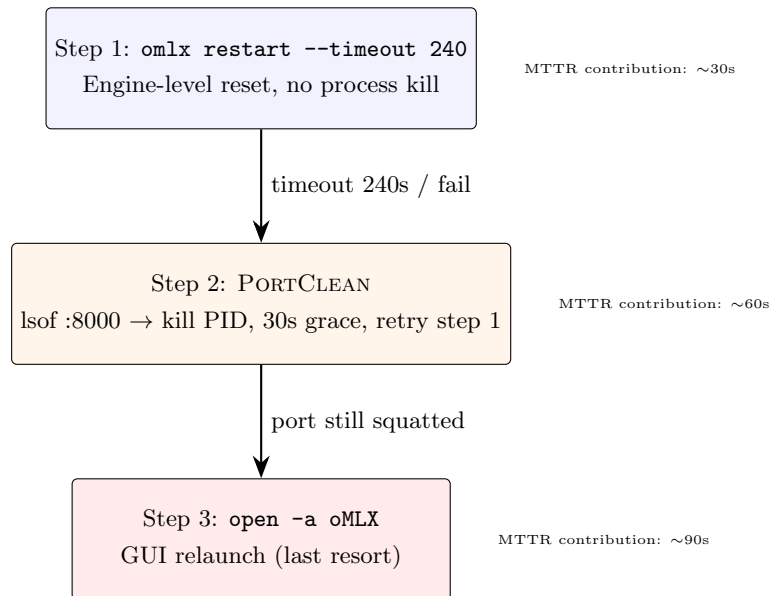


Figure 4: Bounded escalation ladder with timing bounds. Each step is attempted at most once per tick; the next tick re-evaluates from a clean state.

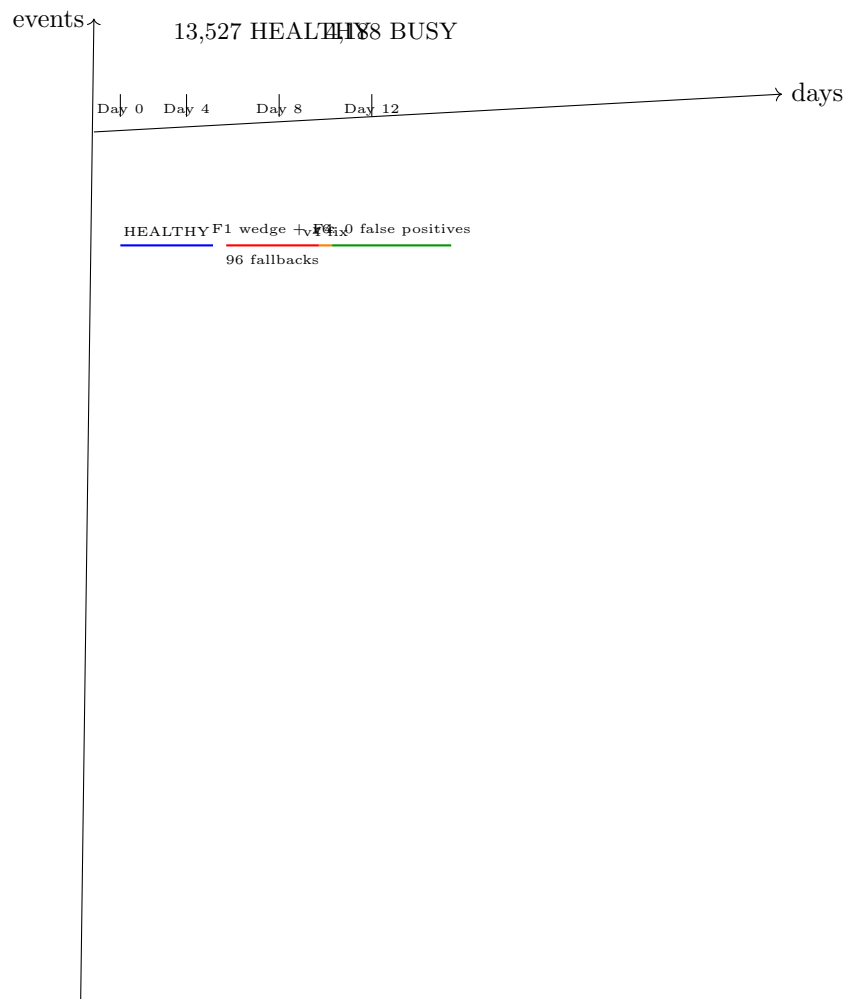


Figure 5: Event timeline across the 13-day trace. The Sep 8 window (F1+F4) shows the ladder under load; post-v6, zero false-positive restarts.